基于 eBPF 的端网协作网络协议实践

沈典

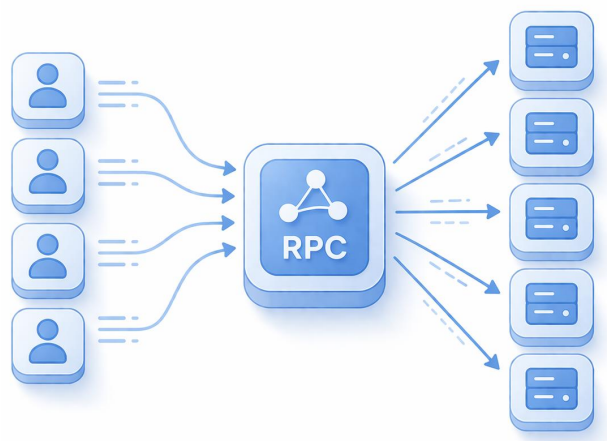
东南大学计算机科学与工程学院

陕西 · 西安　　2026年4月18日

背景

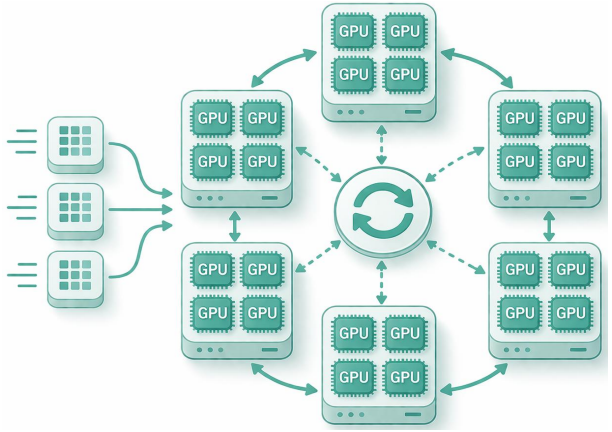
物联网、数据中心、智算中心等新兴网络场景

RPC微服务



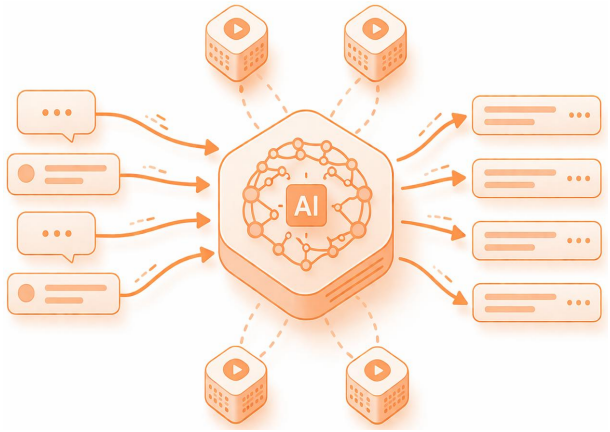
短流密集　多跳调用　尾延迟敏感

分布式模型训练



大吞吐　强同步　慢节点拖累

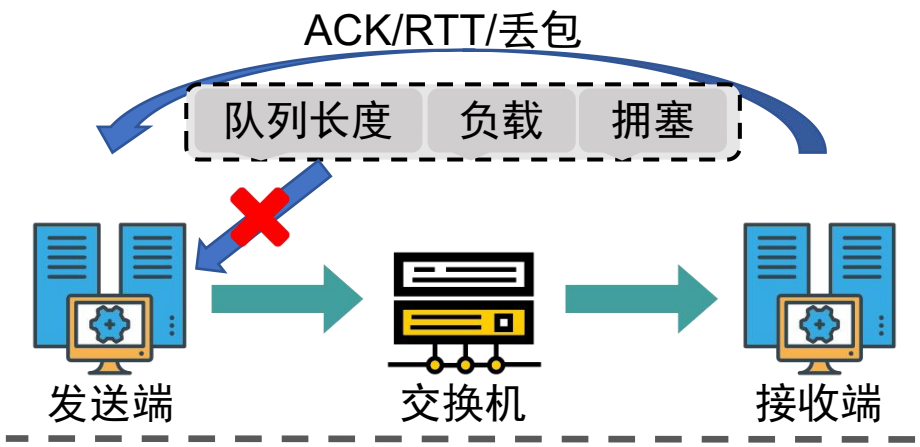
大模型推理

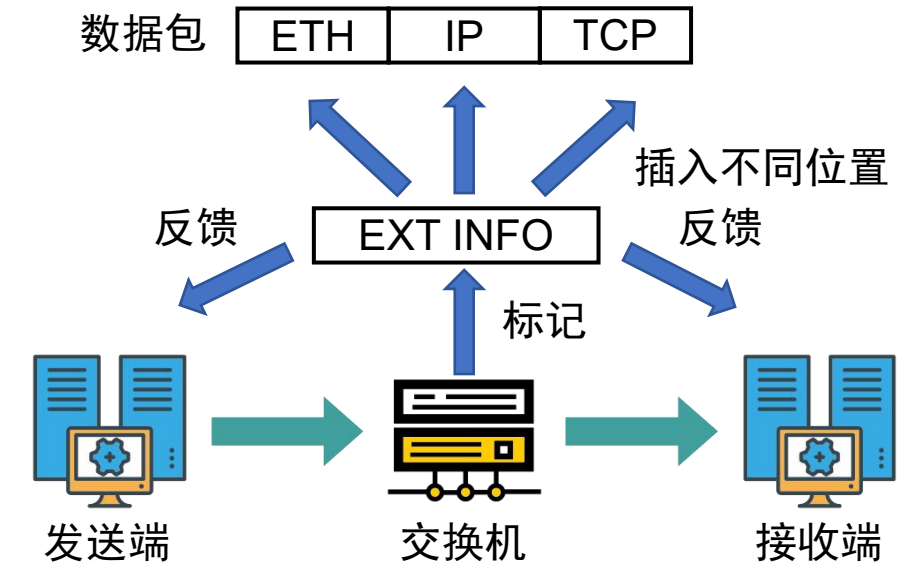


请求突发　流量混合　时延吞吐敏感

网络系统共同的性能诉求：低时延、低抖动、高吞吐

端网协作网络协议（以拥塞控制为例）





传统端到端

反馈慢
需要根据丢包等信号进行调控

网络侧信号不可见
需要根据端侧反馈间接推断网络状态

难以应对突发拥塞
难以及时感知并抑制瞬时队列积压

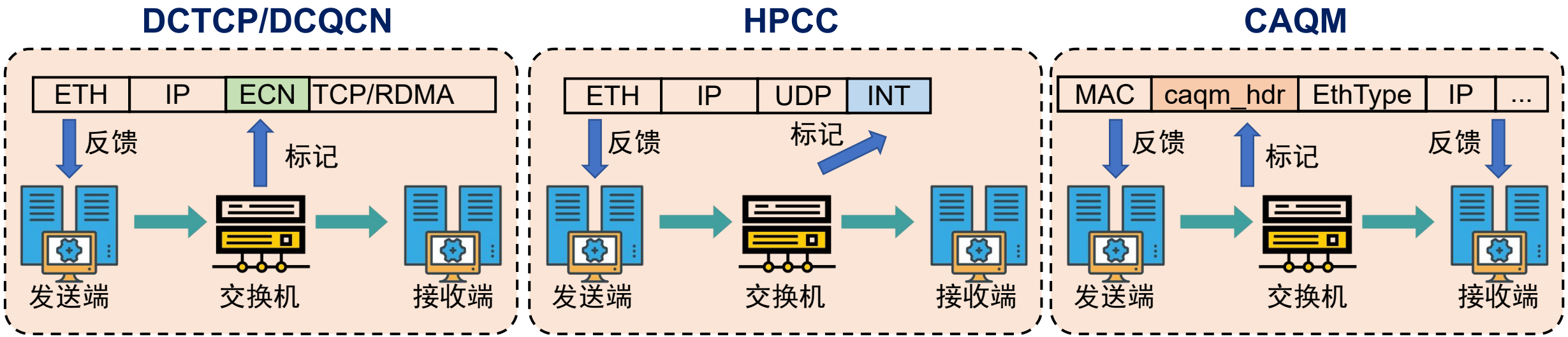
端网协作

反馈快
网络侧信号通过标记快速传递到端侧

网络侧信号可见
发送端可利用网络侧拥塞状态信息

低时延高吞吐
可更及时更准确地实现拥塞控制

端网协作网络协议（以拥塞控制为例）



基于ECN

位置

- IP Header 的 ECN 位

内容

- CE 标记，仅表示是否拥塞

特点

- 开销低、易部署，但表达能力有限

基于遥测

位置

- L4 后的 INT 元数据

内容

- 队列、利用率、时间戳等路径状态

特点

- 信息精细，但开销随路径增长

基于头部扩展

位置

- ETH Header 扩展字段

内容

- c-bit / i-bit标记正负反馈hint标记

特点

- 表达灵活，但会改变二层头结构

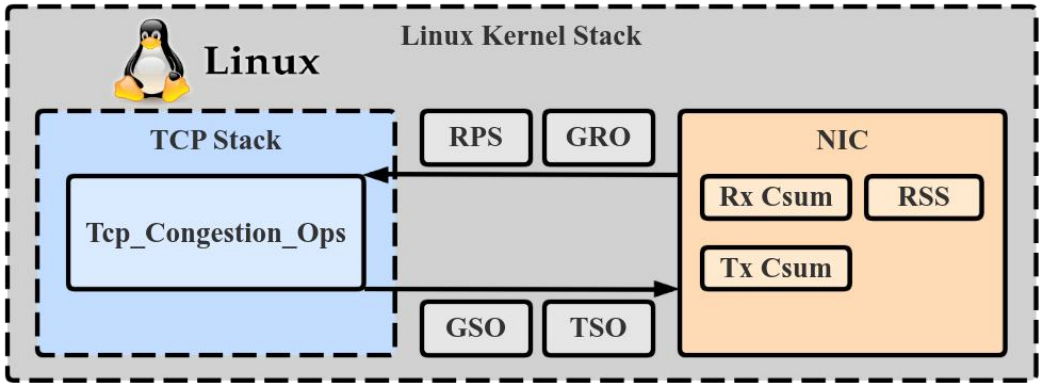
端网协作网络协议的部署问题

以 DCTCP 大规模部署为例：协议虽好，但部署并不容易

Meta, NSDI’24: A large-scale deployment of DCTCP

已有 Linux datapath 深度绑定优化机制

新协议部署不仅是替换 CCA，还会牵动 TCP 栈、NIC 与现有优化路径



Linux协议栈机制

部署收益

- 重传率下降约 **75%**
- P90/P99 延迟下降约 **38%**
- 关闭 DCTCP：吞吐下降 **10%**，重传率上升 **4.5×**

部署挑战

- DCTCP2010年发表，2014年进入Linux内核，2018年Meta尝试大规模部署
- 实践发现大量部署的问题：内核Bug、和其他协议共存、交换机不支持、长链接的替换问题
- 未知的异常、发生异常的回退

端网协作网络协议的部署要求灵活性、稳定性、可靠性

eBPF 为可部署的端网协作协议提供了新的可能

eBPF 带来的新能力

- 安全内核执行：在标准 Linux 内核内扩展协议逻辑
- 无需改内核源码：降低试验与上线门槛
- JIT 高性能：适合在线网络处理
- 多层 Hook 支持：覆盖 XDP、TC、sockops、STRUCT_OPS

为什么“可编程”还不够

部署的核心挑战有以下三点：

- **跨 Hook 协同难**：不同 Hook 看到的状态不同
- **datapath 兼容难**：易与 GRO、RPS、offload 等机制冲突
- **跨版本稳定性不足**：helper、map、hook 接口持续演进

现有工作的启发与局限

现有一些基于eBPF的相关工作：

- Electrode / DINT / BMC / eTran：展示了 eBPF 的网络扩展能力
- 多数工作：更关注功能实现，而非长期部署抽象
- eTran：对协议的深度重构，部署侵入性较强

核心问题

eBPF 提供了协议扩展能力，
但仍缺少面向部署的统一抽象。

相关工作

Zhou et al., Electrode: Accelerating Distributed Protocols with eBPF, **NSDI 2023**.

Zhou et al., DINT: Fast In-Kernel Distributed Transactions with eBPF, **NSDI 2024**.

Ghigoff et al., BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing, **NSDI 2021**.

Chen et al., eTran: Extensible Kernel Transport with eBPF, **NSDI 2025**.

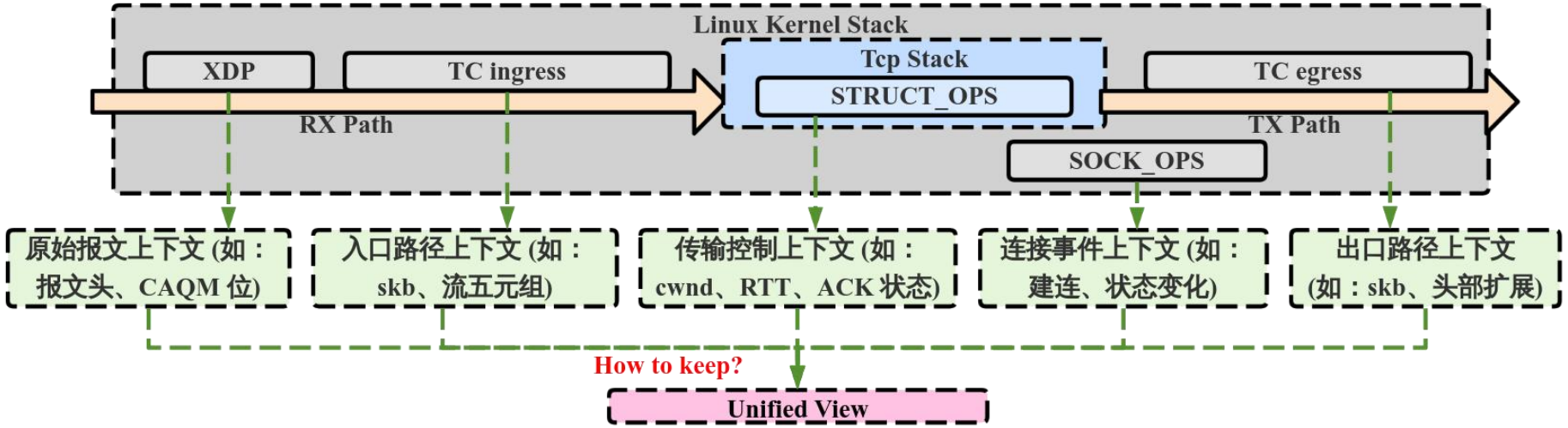
基于 eBPF 部署拥塞控制面临的挑战

碎片化的 Hook 破坏语义对齐

不同 Hook 的状态抽象各异：

- XDP 看包不看连接
- TC 看路径不看控制
- sock_ops 看事件不看逐包
- struct_ops 看控制不看原始信号

Hook	可见信息	不可见信息	局限
XDP	原始报文、逐包信号	socket / flow 状态	看得见包，看不见连接
TC	skb 与数据路径信息	完整传输控制状态	看得见路径，看不见控制
sock_ops	连接事件、部分 socket 上下文	高频逐包信号	看得见事件，看不见逐包
struct_ops	cwnd 等传输控制状态	原始 packet-level 信号	看得见控制，看不见原始信号

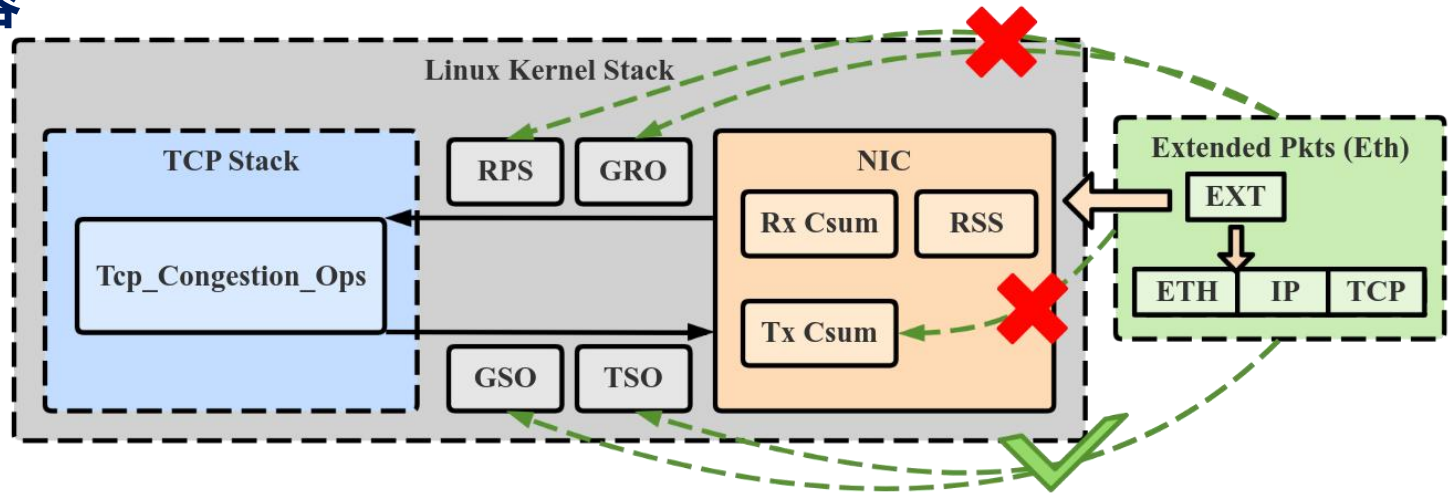


基于 eBPF 部署拥塞控制面临的挑战

与现有协议栈机制的数据路径不兼容

头部扩展可能破坏现有机制，影响兼容性、性能与正确性：

- 头部扩展会被 GRO 破坏逐包语义
- 非标准以太网头部会使 RPS 失效
- 头部扩展导致硬件见校验和失效丢包



现有机制	依赖假设	冲突点	影响
GRO	同流报文可安全聚合	聚合会覆盖逐包拥塞信号	语义失真，影响正确性
RPS	标准 L2 布局可正确解析五元组	非标准头部扩展破坏 flow dissection	导流失效，性能下降
TX checksum offload	NIC 可按固定偏移定位校验字段	头部扩展改变校验和偏移	校验错误，甚至丢包

基于 eBPF 部署拥塞控制面临的挑战

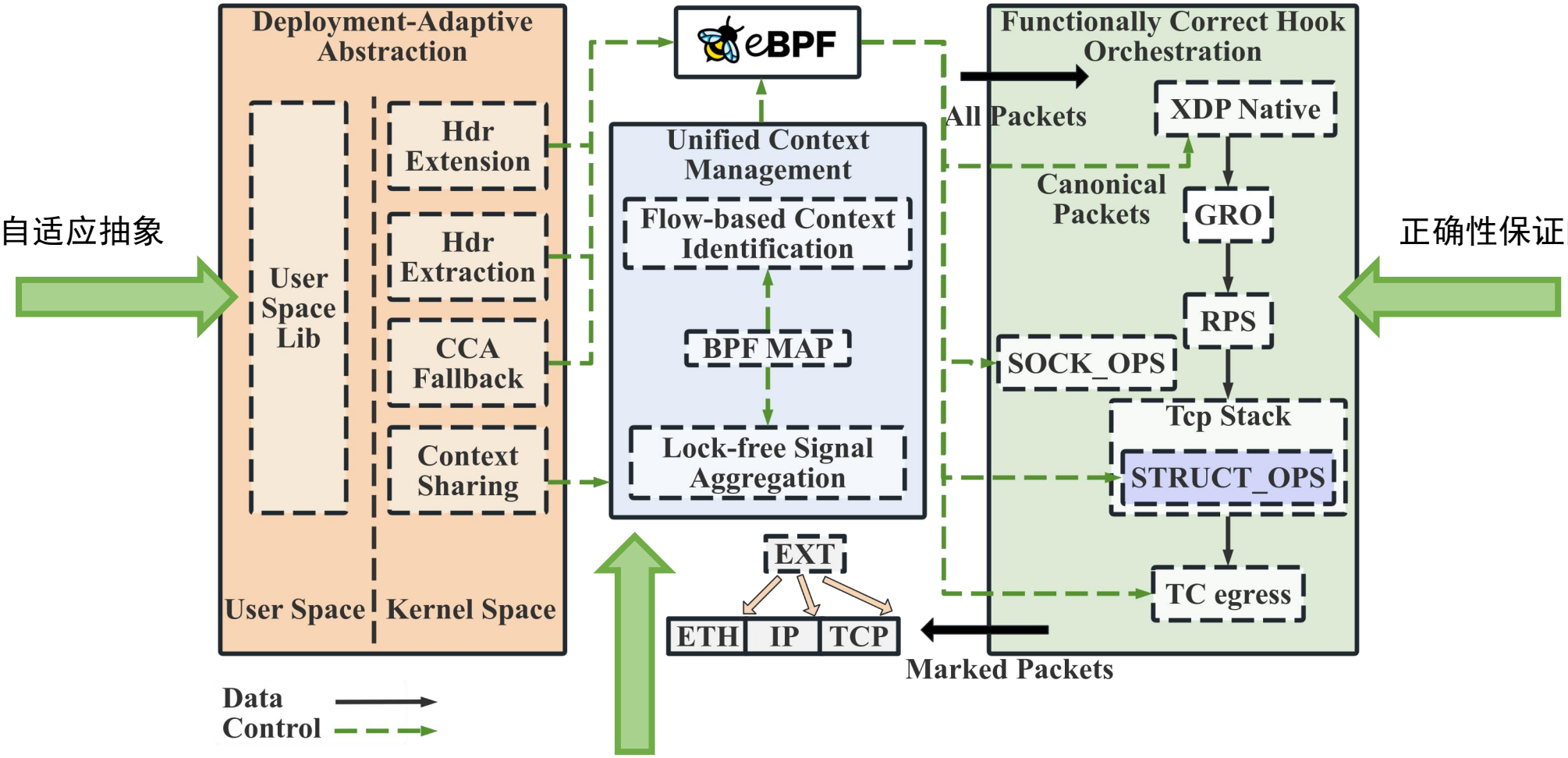
API 演进影响部署稳定性

同一类 CCA 功能，在不同内核版本上往往需要不同 eBPF API 或 Hook 组合实现。例如**回退**：新内核可通过 STRUCT_OPS + bpf_setsockops 切换 CCA，而旧内核通常只能通过 sock_ops 或 TC 触发 fallback

- 若直接绑定**旧机制**，fallback 只能间接触发、表达能力受限
- 若直接绑定**新机制**，又会受内核版本约束，难以兼容旧环境

功能语义	新内核中的实现	旧内核中的实现	部署问题
程序挂载	通过 bpf_link 持久化挂载程序	依赖用户态 daemon 持有 program handle	**挂载**与**生命周期管理**方式不同，运维逻辑需要区分**内核版本**
头部扩展	通过 helper function 插入 TCP options / 扩展字段	手动按字节写入 option bytes	同样的“**加字段**”功能，实现复杂度与可维护性差异很大
头部解析	使用 helper function 解析 TCP 选项 / 头部	依赖字节级手工解析	**解析逻辑**更易受**内核接口变化**影响，代码可移植性差
上下文共享	支持 fetch_and 等更完整原子操作	只能用较弱的简化原子更新	**锁无关聚合**与 **producer-consumer 语义**难以统一复用
CCA fallback	通过 STRUCT_OPS + bpf_setsockops 切换 CCA	退化到 sock_ops 或 TC 触发 fallback	相同“fallback”语义，在**不同内核**上依赖完全**不同 Hook 路径**

eCCA-一种基于eBPF的拥塞控制协议实现方案



eCCA 设计一：统一上下文管理

在 Hook 语义碎片化的前提下，eCCA 通过共享上下文实现跨 Hook 协同

跨 Hook 状态交换必须显式完成

- eBPF 程序只能直接访问 Hook 局部状态
- 跨 Hook 信息交换必须通过 BPF map
- eCCA 在 BPF map 之上构造 shared context abstraction

共享上下文为什么会并发冲突

- 跨 Hook 状态交换必须通过 BPF map
- 不同 Hook 可能在不同 CPU上执行
- 同一 flow context 可能被并发读写
- 例如开启 RPS ，native XDP 与 STRUCT_OPS 可能跨 CPU 并发访问同一 flow context，导致读写不一致

基于流的上下文标识

- 以五元组作为上下文键
- CCA 决策通常本就以流为粒度
- 不同流状态天然隔离，可减少跨流竞争
- RSS / RPS / RFS 往往会把同一流导向同一 CPU

无锁信号聚合

观察

- 多数拥塞信号是可聚合数值
- spinlock 开销高，不适合高速路径

设计

- 原子加 / 读改写可支持高效协同
- 提取阶段累计，决策阶段读取并清零

eCCA 设计二：正确性保证的Hook编排

选择同时保持算法正确性和 Linux datapath 兼容性的内核 Hook 编排方式

GRO

冲突：GRO 聚合会丢失逐包语义
编排：在 native XDP 提取信号
效果：保留逐包语义

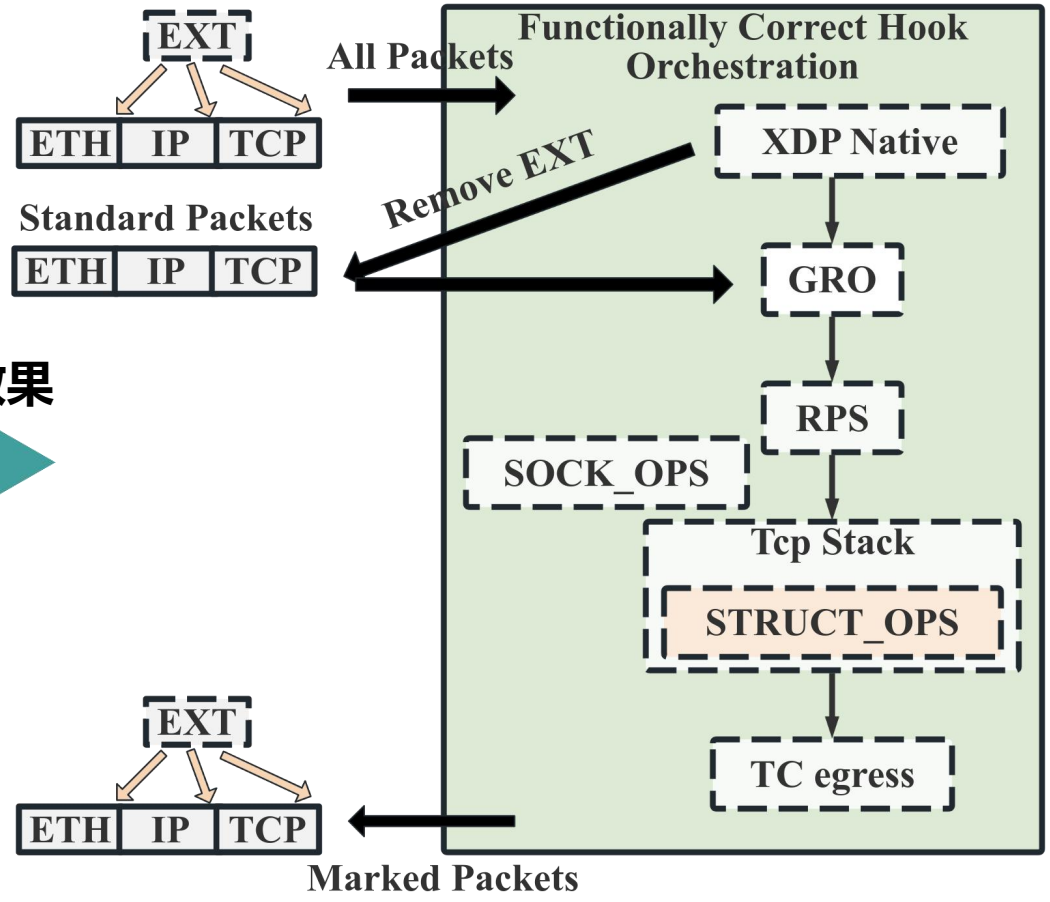
RPS

冲突：ETH 头部扩展破坏标准 L2 布局
编排：在 RPS 前移除扩展头
效果：保持转发与导流有效

tx-checksum

冲突：头部扩展改变校验和偏移
编排：在受影响网卡关闭 tx-csum
效果：保证校验正确性

编排效果

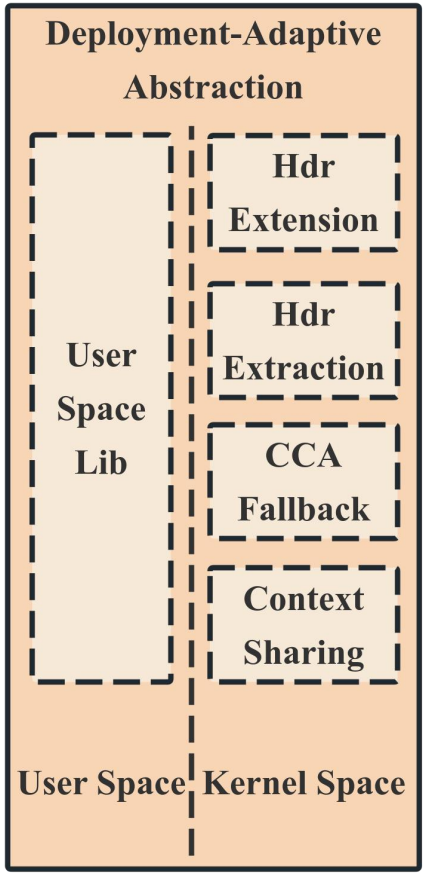


eCCA 设计三：面向可演进部署的自适应抽象

需要将 CCA 与不断演进的内核接口解耦，以保持跨内核版本的可移植性

- **内核会变：** helper、hook、map 等接口跨版本演进
- **语义不变：** CCA 关注的是插头、取头、共享状态与回退控制
- **抽象承接变化：** 用稳定 API 隔离版本相关实现细节
- **部署自适应：** 在不同内核上自动选择可用且更高效的实现路径

组件	代表性 API	描述	兼容性处理
用户态库	attach/detach_program()	程序编排	**新：通过 bpf_link 持久化挂载；** 旧：由用户态守护进程持有程序句柄
头部扩展	ext_eth/iph/tcph()	插入 CCA 信号	**新：通过 helper 插入 TCP 选项；** 旧：手动写入 TCP option 字节
头部提取	extract_eth/iph/tcph()	提取 CCA 信号	**新：通过 helper API 解析 option；** 旧：按字节解析 TCP 头部
上下文共享	producer/consumer()	无锁状态共享	**新：支持 fetch_and 等原子操作；** 旧：不带返回值的简化原子更新
CCA 回退	cca_enable/status() / cca_fallback()	CCA 回退控制	**新：通过 STRUCT_OPS 与 bpf_setsockops 切换 CCA；** 旧：通过 sock_ops 或 tc 触发回退



案例：CAQM-e

在 eCCA 上实现 CAQM，在真实系统上进行验证

CAQM-e

原始 CAQM

- 在以太网头中插入 4 字节 caqm_hdr 携带拥塞信号
- 已部署于商用交换机

实践问题

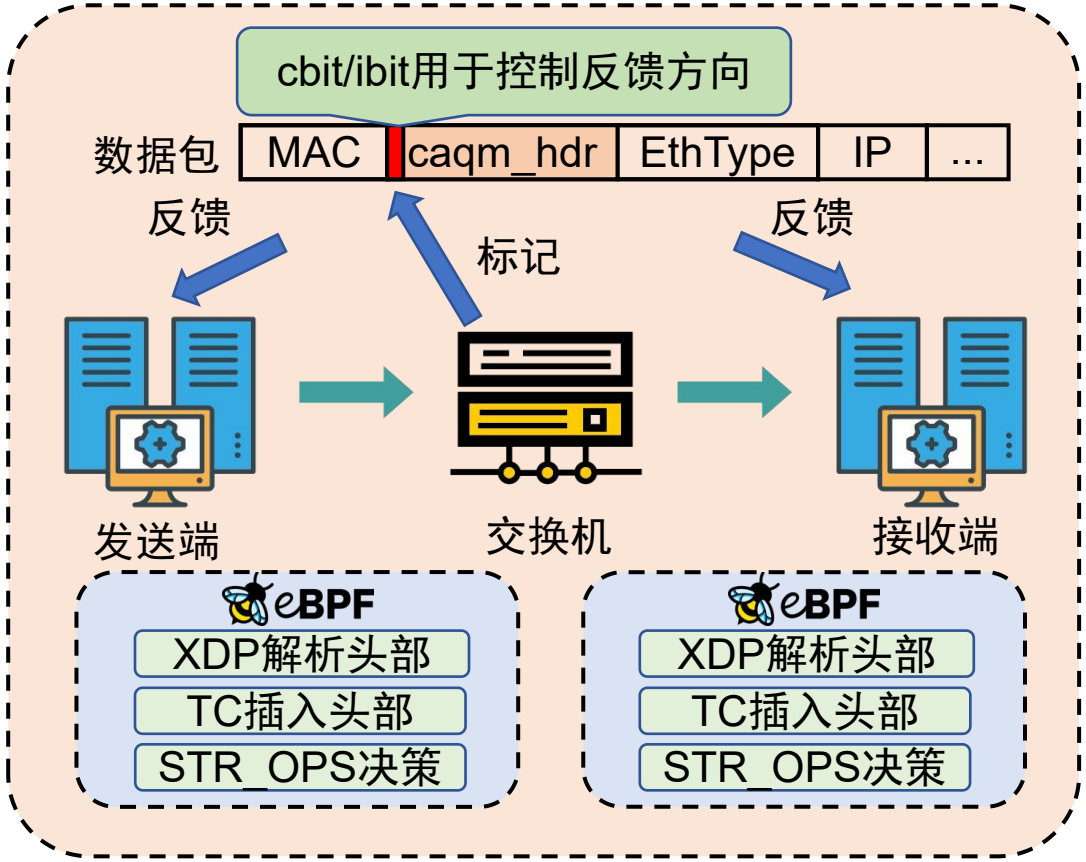
- 监听 socket 不经过连接建立回调
- 监听 socket 持续引用 bpf_caqm
- 引用导致 STRUCT_OPS 无法卸载

最终方案

仅对通用服务端口在 TCP_LISTEN_CB 阶段回退 Reno，应用端口保持 bpf_caqm，**兼顾安全卸载与正常握手**

工作流程

CAQM-e 工作流程如右图所示TC 插头、交换机标记、XDP 提取、STRUCT_OPS 调窗



利用eCCA重构CAQM协议，实现交换机反馈驱动的调窗闭环

实验环境

Q1：eCCA 能否保持原始控制语义？
Q2：CAQM-e 能否实现低尾时延与公平性？
Q3：eCCA 的部署优势与额外开销如何？

Baselines

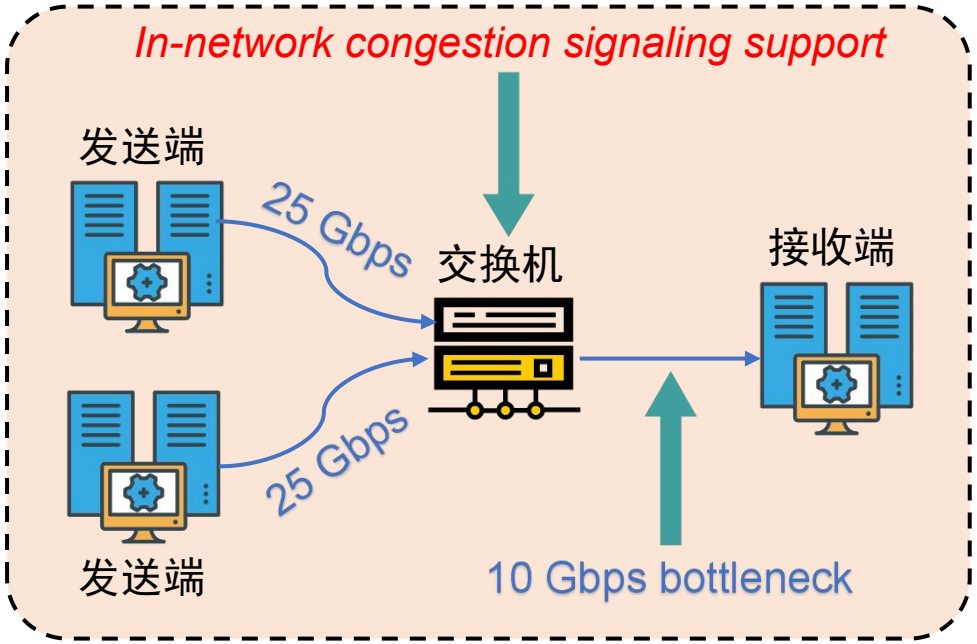
- CUBIC
- 原生 CAQM
- CAQM-e

Deployment Scope

依赖 native XDP 与 STRUCT_OPS（Linux ≥ 5.6），已在 Linux 5.6–6.16 及 Intel / Mellanox NIC 上验证

Testbed

发送端(OpenEuler)
Xeon E5-2690 v3 + 2 × Intel 25GbE
接收端(OpenEuler)
Kunpeng-920 + Mellanox 25GbE

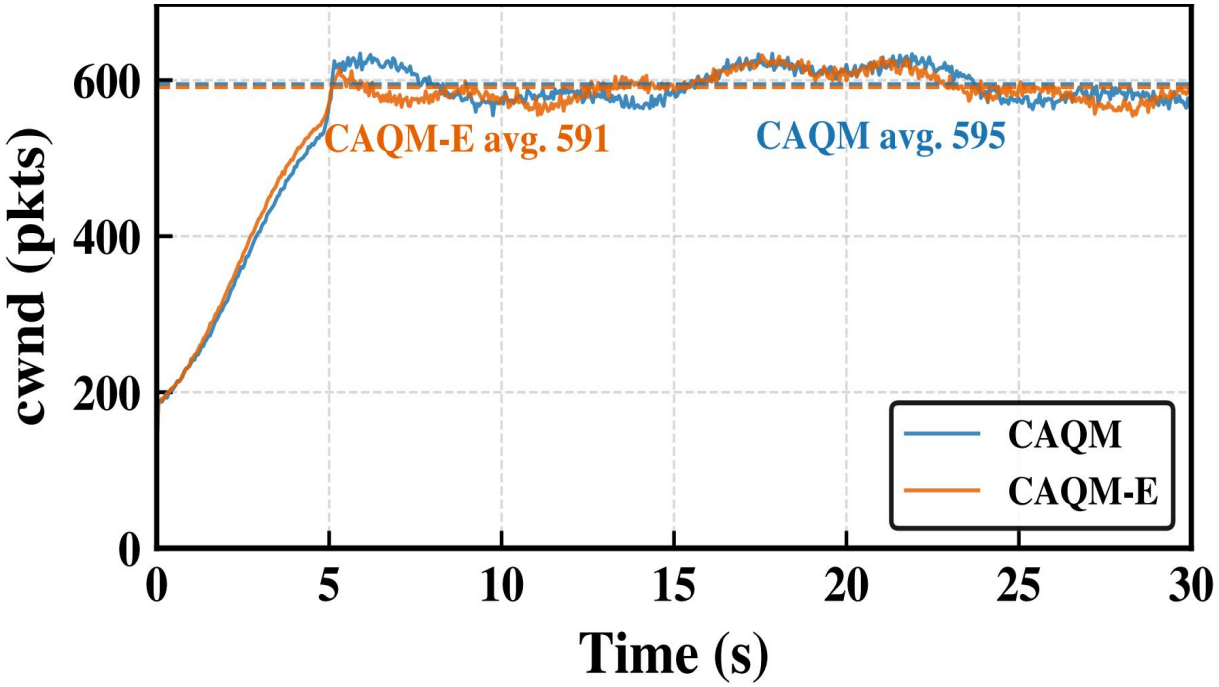


Q1-实验结果

eCCA 能否保持原始算法的控制语义

eCCA算法语义保真性

- 在相同流量模式下，对比 CAQM 与 CAQM-e 的 cwnd 演化轨迹
- CAQM-e 在收敛速度、振荡幅度、稳态窗口上均与原生 CAQM 高度一致
- 平均 cwnd 分别为 591 pkts（CAQM-e） 与 595 pkts（CAQM），差异极小
- 说明 eCCA 虽将逻辑分布到 TC/XDP/STRUCT_OPS，仍能保持原始算法的控制保真性



eCCA 保留了 CAQM 的拥塞控制语义，实现与CAQM一致的调控效果

Q2-实验结果

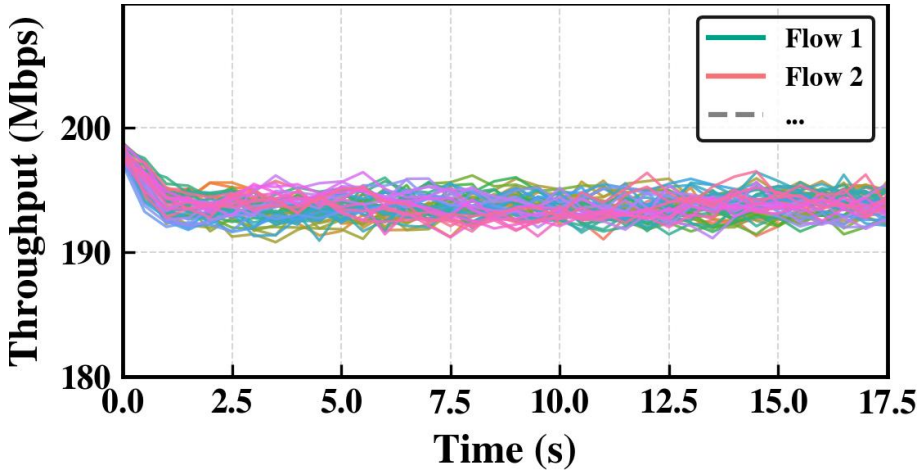
CAQM-e 在真实部署下能否实现低尾时延与公平性

低尾时延

CC	2-to-1		3-to-1		4-to-1	
	Tail Lat.	Loss	Tail Lat.	Loss	Tail Lat.	Loss
CUBIC	4033.5	Yes	4043.9	Yes	4203.0	Yes
CAQM	152.3	No	137.0	No	175.8	No
CAQM-e	132.9	No	140.0	No	173.2	No

在 2-to-1 / 3-to-1 / 4-to-1 incast 场景下，CAQM 和 CAQM-e 均实现了亚 200 μs 尾延迟且零丢包，而 CUBIC 出现了毫秒级尾时延和丢包

公平性



在 50-to-1 incast 场景下，CAQM-e 保持了稳定公平性，各流吞吐维持在 192–196 Mbps 之间，与原生 CAQM一致

Q3-实验结果

使用eCCA的优势与带来的额外性能开销

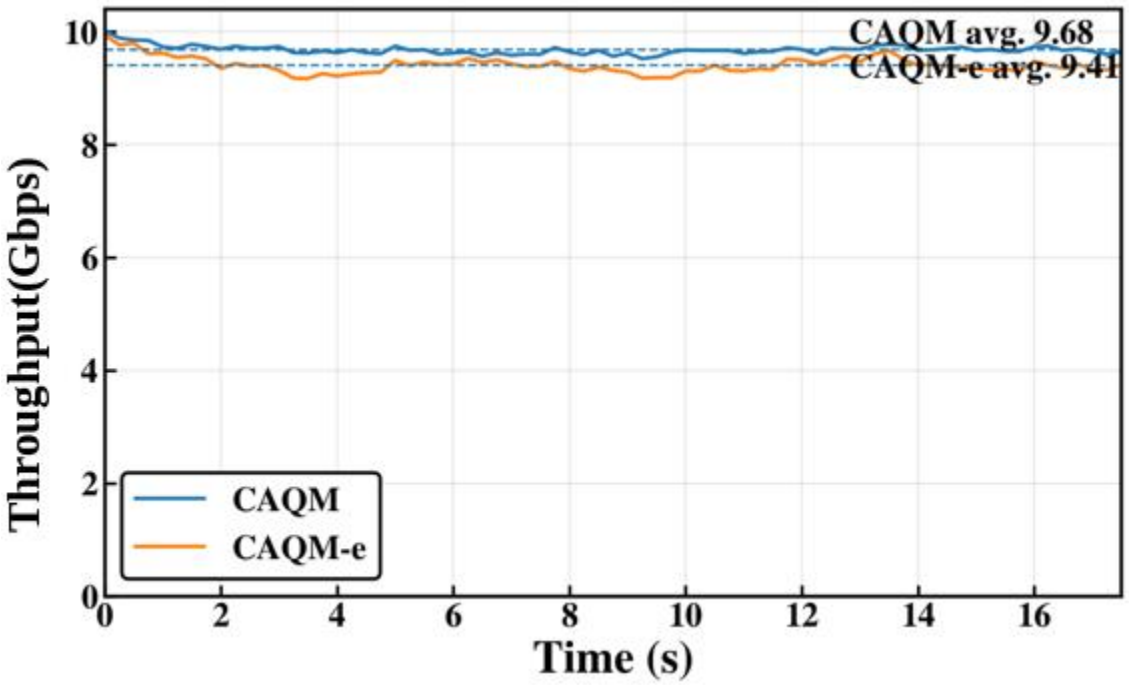
eCCA优势

- 通过共享上下文可以动态调整参数
- 通过Hook编排兼容内核现有机制
- 通过部署自适应抽象与内核版本解耦

eCCA 额外开销如何

- 吞吐量由 9.68 Gbps 降至 9.41 Gbps
- eCCA 带来约 2.8% 吞吐量开销，如 map 读写，数据包解析等

在保留部署灵活性的同时，仅引入约 2.8% 的额外开销



eCCA 兼具良好部署性与低额外开销，仅引入较小的额外开销

展望

更多协议

- 基于ECN的协议（如DCTCP）
- 基于遥测的协议（如HPCC）
- 基于头部扩展的协议

eCCA 有望为多类端网协作协议提供统一的实现与部署底座

更多场景

- RPC 微服务
- 分布式训练
- 大模型推理

eCCA 有望支撑端网协作协议按场景部署与优化

更大规模部署

- 多内核版本
- 多 NIC 环境
- 多交换机平台

eCCA 有望推动端网协作协议从原型验证走向规模化部署

谢谢！